

Análisis sobre el Dataset de Indicadores de Ocupabilidad del Ministerio de Comercio Exterior y Turismo

Estudiantes:

Sergio Sebastian Pezo Jimenez
Universidad Nacional de Ingeniería,
Facultad de Ciencias
e-mail: sergio.pezo.j@uni.pe

Arbués Enrique Perez Villegas
Universidad Nacional de Ingeniería,
Facultad de Ciencias
e-mail: arbues.perez.v@uni.pe

Curso:

CC531 Análisis en Macrodatos
Laboratorio 01

Resumen

Se trabajó con los indicadores mensuales de ocupabilidad hotelera de 2019 a junio de 2025. Primero se unificaron siete archivos CSV y se obtuvo una entrada canónica de 38 730 registros. Luego se ejecutaron catorce consultas Java sobre Hadoop 3: diez consultas de análisis, dos modelos de clasificación y dos modelos de regresión. Se logró conservar la salida y el archivo JAR de cada consulta. También se revisaron HDFS, YARN, JobHistory y el consumo de los contenedores. En clasificación, la regresión logística obtuvo mejores resultados generales que Gaussian Naive Bayes. En regresión, ridge tuvo una mejora pequeña frente al modelo lineal en la validación de 2025-I.

Palabras clave: Hadoop, MapReduce, ocupabilidad, clasificación, regresión.

Índice

1	Introducción	3
1.1	Contexto del problema	3
1.2	Objetivos del laboratorio	3
1.3	Organización del informe	3
2	Entorno Experimental y Despliegue de Hadoop	3
2.1	Arquitectura del entorno	3
2.2	Configuración de la máquina virtual de nodo único	3
2.3	Despliegue de Hadoop 3 mediante Docker Compose	3
2.4	Servicios de Hadoop y persistencia de datos	3
2.5	Configuración de HDFS y ejecución de MapReduce	4
2.6	Verificación del entorno	4
3	Descripción del Dataset	4
3.1	Fuente, publicación y actualización	4
3.2	Descripción general del dataset	4
3.3	Estructura, campos y tipos de datos	4
3.4	Diccionario de las 24 columnas	4
3.5	Qué significa el filtro TT/TT	5
3.6	Preprocesamiento y carga en HDFS	5

3.7	Antecedentes e investigaciones relacionadas	6
3.7.1	Investigaciones previas sobre el dataset	6
3.7.2	Artículos que utilizaron el dataset	6
4	Metodología	6
4.1	Proceso de procesamiento distribuido	6
4.2	Diseño de preguntas y consultas	6
4.3	Flujo de ejecución	6
4.4	Métricas de evaluación	7
5	Consultas MapReduce	7
5.1	Resumen de consultas y cantidad de etapas	7
5.2	Consultas que emplean dos o más campos	7
5.2.1	Consulta 1: arribos nacionales y extranjeros	7
5.2.2	Consulta 2: TNOH por departamento y año	8
5.2.3	Consulta 3: TNOH y TNOC por clase	8
5.2.4	Consulta 4: empleo por capacidad	8
5.2.5	Consulta 5: estacionalidad mensual	8
5.3	Medidas estadísticas	9
5.3.1	Consulta 6: media, mediana y desviación estándar de TNOH	9
5.4	Búsqueda de subtexto	9
5.4.1	Consulta 7: búsqueda sobre campos de texto	9
5.5	Valores máximos y mínimos	9
5.5.1	Consulta 8: extremos de TNOH por año	9
5.6	Consultas con MapReduce encadenado	10
5.6.1	Consulta 9: cambio de ocupación entre 2019 y 2024	10
5.6.2	Consulta 10: participación de arribos extranjeros	10
6	Consultas mediante Modelos de Clasificación	10
6.1	Consulta 11: Gaussian Naive Bayes	10
6.2	Consulta 12: regresión logística	11
6.3	Comparación de modelos de clasificación	11
7	Consultas mediante Modelos de Regresión	11
7.1	Consulta 13: regresión lineal	11
7.2	Consulta 14: regresión ridge	11
7.3	Comparación de modelos de regresión	12
8	Monitoreo de Recursos	12
8.1	Recursos observados durante la ejecución	12
8.2	Consumo de CPU	13
8.3	Consumo de memoria	13
8.4	Tiempo de ejecución	13
8.5	Análisis del comportamiento de Hadoop	14
9	Resultados y Discusión	14
9.1	Resultados de las consultas MapReduce	14
9.2	Resultados de clasificación	14
9.3	Resultados de regresión	14
9.4	Discusión general	14
10	Conclusiones	14
A	Anexo: Código fuente Java y archivos JAR	14
B	Anexo: Inputs y outputs de las consultas	15
C	Anexo: Docker Compose y configuración de Hadoop	15

D Anexo: Documentación adicional	15
Bibliografía	16

1. Introducción

1.1. Contexto del problema

Los indicadores de hospedaje cambian por mes, departamento y tipo de establecimiento. Al juntar varios años aparecen diferencias de estructura, valores repetidos por nivel de agregación y un volumen que ya no resulta cómodo para revisar fila por fila. La revisión de trabajos anteriores mostró que estos datos se habían usado sobre todo en estudios regionales y con métodos estadísticos clásicos. Por eso se usó Hadoop MapReduce para trabajar todos los departamentos con un flujo repetible. El objetivo no fue solo obtener números, sino relacionar cada decisión de procesamiento con un problema real del dataset y revisar cada salida con datos reales.

1.2. Objetivos del laboratorio

Se buscó consolidar los archivos de 2019 a 2025-I, cargar una única entrada a HDFS y resolver catorce preguntas con Java MapReduce. También se compararon dos clasificadores y dos regresores usando una separación temporal. Finalmente, se registró el estado del clúster y el uso de recursos durante una ejecución.

1.3. Organización del informe

Primero se explica el entorno y la preparación del dataset. Después se muestra la metodología y las diez consultas de análisis. Luego se presentan los modelos de clasificación y regresión. Al final se revisan el monitoreo, los resultados principales, las limitaciones y las conclusiones.

2. Entorno Experimental y Despliegue de Hadoop

2.1. Arquitectura del entorno

Se usó un clúster Hadoop 3.5.0 en Docker Compose. El despliegue tuvo un NameNode, un DataNode, un ResourceManager, un NodeManager y un HistoryServer. Todos los servicios se ejecutaron en una sola máquina. Así se logró probar HDFS y YARN sin presentar el entorno como un clúster de producción.

2.2. Configuración de la máquina virtual de nodo único

El NodeManager informó 2 GB de memoria física para contenedores y 8 vcores. El nodo se mantuvo saludable. Como solo hubo un nodo, los mapas y reducers compartieron el mismo equipo. Esto comprueba el flujo, pero no permite medir escalamiento horizontal.

2.3. Despliegue de Hadoop 3 mediante Docker Compose

Los contenedores ya estaban definidos en `hadoop3/compose.yaml`. Se inició el entorno con el script del proyecto y se hizo una prueba rápida antes de ejecutar las consultas. No se reconstruyeron imágenes. Se usaron los servicios existentes y sus volúmenes persistentes.

2.4. Servicios de Hadoop y persistencia de datos

HDFS guardó la entrada y las salidas. YARN administró las aplicaciones MapReduce. JobHistory conservó el detalle de los trabajos terminados. El NameNode mostró 203 bloques replicados y un DataNode activo.

2.5. Configuración de HDFS y ejecución de MapReduce

Se usó `hadoop3/scripts/run-job.sh`. El script copia el código al NameNode, compila las clases Java, arma el JAR, sube el CSV a HDFS y ejecuta el Driver. Si la salida termina con `_SUCCESS`, exporta el resultado y el JAR al host. Para repetir todo se creó `labs/labca1/queries/run-all.sh`.

2.6. Verificación del entorno

Las catorce consultas terminaron con estado `SUCCESS`. Se observaron 33 aplicaciones finalizadas porque las consultas 9 y 10 usan dos etapas y también quedaron ejecuciones de prueba anteriores. JobHistory confirmó que la consulta 14 lanzó dos mapas y un reducer sin intentos fallidos.

3. Descripción del Dataset

3.1. Fuente, publicación y actualización

El dataset de Indicadores de Ocupabilidad fue publicado por el Ministerio de Comercio Exterior y Turismo (MINCETUR) en la Plataforma Nacional de Datos Abiertos. La fuente reúne estadísticas mensuales sobre la capacidad y el uso de los establecimientos de hospedaje del país. El registro fue publicado en 2021 y su metadato tuvo una actualización en agosto de 2026 [1].

La fuente no se eligió de forma arbitraria. El Ministerio de Economía y Finanzas recomienda estos indicadores como referencia para estimar demanda en proyectos públicos de turismo [2]. Además, la encuesta que los produce tiene respaldo del INEI y usa censos o muestras según la categoría y la zona estudiada [3]. Esto permitió trabajar con una fuente oficial y también reconocer sus límites.

3.2. Descripción general del dataset

El archivo consolidado cubre desde enero de 2019 hasta junio de 2025. Contiene 38 730 registros y 24 columnas. Se encontraron 25 departamentos en cada mes. El año 2025 se trató como primer semestre y no como un año completo.

3.3. Estructura, campos y tipos de datos

Los campos principales son año, mes, clase, categoría, departamento, cantidad de establecimientos, habitaciones, plazas-cama, TNOH, TNOG, arribos, pernoctaciones y empleo. Los archivos de 2019 a 2023 tenían 23 columnas. Los archivos recientes añadían `FECHA_CORTE`. La cabecera final usa nombres sin espacios y mantiene todos los indicadores en un orden fijo.

3.4. Diccionario de las 24 columnas

Cada fila representa un mes, departamento y nivel de desagregación por clase y categoría del hospedaje. Por ello, una fila no representa un hotel individual. Los campos de capacidad y actividad son agregados del grupo definido por `ANIO`, `MES`, `DEPARTAMENTO`, `ID_CLASE` e `ID_CATEGORIA`. El siguiente cuadro adapta el diccionario oficial de MINCETUR [1] y aclara cómo se interpretó cada columna en este laboratorio.

Cuadro 1: Diccionario de columnas del CSV consolidado

Columna	Significado e interpretación
<code>FECHA_CORTE</code>	Fecha hasta la cual fue actualizada o cerrada la información de la fuente. Está vacía en los archivos antiguos y no es el mes observado.
<code>ANIO</code>	Año al que corresponde la observación mensual. El valor 2025 solo cubre enero-junio.
<code>MES</code>	Mes de referencia, codificado de 01 a 12. Se conserva con dos dígitos para ordenar los periodos correctamente.
<code>ID_CLASE</code>	Código de la clase del hospedaje: por ejemplo, 01 hotel, 02 apart hotel, 03 hostel o TT total consolidado.
<code>CLASE</code>	Nombre legible de la clase. Cuando el código es TT, el valor es <i>TODAS CONSOLIDADAS</i> : reúne todas las clases del departamento.

Columna	Significado e interpretación (continuación)
ID_CATEGORIA	Código de la categoría: 01–05 para una a cinco estrellas, otros códigos específicos o TT para el total.
CATEGORIA	Nombre legible de la categoría. <i>NO CATEGORIZADO</i> no significa dato perdido: identifica establecimientos sin categoría de estrellas.
DEPARTAMENTO	Departamento o región del Perú donde se agrega la información. El dataset contiene 25 valores, incluyendo Callao.
ID_UBIGEO	Código geográfico de seis caracteres del INEI. En estas filas departamentales termina normalmente en cuatro ceros; debe tratarse como texto para no perder ceros iniciales.
NUMERO_ESTABLECIMIENTOS	Cantidad mensual de establecimientos de hospedaje incluidos en el grupo de la fila.
NUMERO_HABITACIONES	Cantidad de habitaciones ofertadas por esos establecimientos; mide capacidad física, no habitaciones ocupadas.
NUMERO_PLAZAS_CAMA	Capacidad de alojamiento expresada en camas disponibles; una cama doble equivale a dos plazas-cama.
PORCENTAJE_TNOH	Tasa Neta de Ocupación de Habitaciones: porcentaje mensual de la capacidad de habitaciones realmente disponible que fue ocupada. La unidad es porcentaje, no número de habitaciones.
PORCENTAJE_TNOC	Tasa Neta de Ocupación de Plazas-Cama: porcentaje mensual de la capacidad de camas realmente disponible que fue ocupada. Puede diferir de TNOH porque una habitación ocupada puede alojar una o varias personas.
PROMEDIO_PERMANENCIA	Promedio general de días o noches que cada huésped permanece alojado.
PROMEDIO_PERMANENCIA_NAC	Relaciona pernoctaciones con arribos.
PROMEDIO_PERMANENCIA_EXT	Permanencia promedio de los huéspedes nacionales, expresada en días.
TOTAL_ARRIBOS	Permanencia promedio de los huéspedes extranjeros, expresada en días.
TOTAL_ARRIBOS_NAC	Personas registradas al llegar durante el mes. Cada huésped se cuenta una vez al inicio de su estadía, aunque permanezca varias noches.
TOTAL_ARRIBOS_EXT	Parte de los arribos correspondiente a huéspedes nacionales.
TOTAL_PERNOCT	Parte de los arribos correspondiente a huéspedes extranjeros; junto con los nacionales compone el total de arribos.
TOTAL_PERNOCT_NAC	Total de noches ocupadas por todos los huéspedes. No es igual a arribos: una persona que permanece tres noches aporta un arribo y tres pernoctaciones.
TOTAL_PERNOCT_EXT	Pernoctaciones generadas por huéspedes nacionales.
TOTAL_EMPLEO	Pernoctaciones generadas por huéspedes extranjeros.
	Personas empleadas durante el mes por los establecimientos del grupo; no representa horas trabajadas ni calidad del empleo.

3.5. Qué significa el filtro TT/TT

TT no es un valor desconocido: significa *TODAS CONSOLIDADAS*. Una fila TT/TT tiene simultáneamente ID_CLASE=TT e ID_CATEGORIA=TT; por tanto, es el total mensual del departamento sin separar clase ni categoría. El CSV también contiene filas detalladas y subtotales. Sumarlas junto con TT/TT volvería a contar la misma actividad varias veces. Por eso, las consultas de totales, promedios departamentales y modelos usan solo TT/TT. La consulta 3 es la excepción intencional: conserva el total de categoría de cada clase (ID_CATEGORIA=TT) y excluye la clase global para poder comparar hoteles, hostales, resorts y demás clases.

3.6. Preprocesamiento y carga en HDFS

Los archivos no tenían exactamente la misma estructura. Los datos de 2019 a 2023 usaban 23 columnas, mientras que los archivos más recientes añadían la fecha de corte. Por eso, se creó un archivo consolidado con 24 columnas. También se cambió la coma decimal por un punto y se guardó el resultado en UTF-8. Al final, se obtuvieron 38 730 registros entre enero de 2019 y junio de 2025.

Los archivos originales se leyeron como Latin-1 y se conservaron intactos. El script validó la cabecera, la cantidad de columnas, el rango de años y los campos numéricos. Para agregaciones generales se usaron solo las filas totales de clase y categoría, identificadas como TT/TT. Ese filtro fue necesario porque el dataset repite totales en distintos niveles. La investigación previa sobre la fuente permitió detectar dos riesgos antes de diseñar las consultas: el cambio de esquema entre 2023 y 2024 y el doble conteo producido por las filas

consolidadas. Por eso la normalización y el filtro TT/TT quedaron como reglas comunes para las consultas que calculan totales nacionales.

3.7. Antecedentes e investigaciones relacionadas

3.7.1. Investigaciones previas sobre el dataset

Se encontraron dos investigaciones peruanas que usaron los indicadores de ocupabilidad como parte central del análisis. Taípe y Laura estudiaron la llegada mensual de turistas y la oferta de hospedaje en Huancavelica entre 2003 y 2019 mediante regresión lineal y variables estacionales [4]. Ñahuincopa y Rojas también trabajaron Huancavelica, pero analizaron la relación entre demanda turística y capacidad hotelera durante 2010–2019 [5].

Estos antecedentes mostraron que el dataset sí tenía uso académico, aunque el alcance estaba concentrado en una región y en métodos econométricos. Esa diferencia influyó en el diseño del laboratorio. Las consultas 1 a 10 se plantearon para comparar años, meses, departamentos y tipos de hospedaje en todo el país. Las consultas 11 a 14 ampliaron el análisis con dos clasificadores y dos regresores evaluados mediante una división temporal.

3.7.2. Artículos que utilizaron el dataset

No se encontró un artículo que aplicara Hadoop, MapReduce o aprendizaje automático directamente al CSV de ocupabilidad. Sí se encontró un trabajo que combinó datos abiertos de turismo peruano con Google Trends y TripAdvisor para comparar modelos de regresión y árboles, pero usó arribos a sitios turísticos en lugar de ocupabilidad hotelera [6].

También existe un antecedente peruano que levantó una solución Hadoop sobre datos de la misma Plataforma Nacional de Datos Abiertos, aunque en el sector salud [7]. Ese trabajo sirvió como referencia para la parte técnica. En este laboratorio se aplicó la idea al turismo con Hadoop 3.5.0 en un solo nodo, levantado mediante Docker Compose. El aporte se centró en un pipeline reproducible, en la limpieza del esquema y en catorce consultas sobre la desagregación nacional. La literatura también señala que los datos administrados por instituciones públicas se aprovechan menos que otras fuentes turísticas [8], lo que reforzó el uso de una fuente oficial abierta.

4. Metodología

4.1. Proceso de procesamiento distribuido

Cada consulta lee el CSV canónico mediante un parser común. El Mapper descarta la cabecera y las filas que no cumplen el filtro. Después emite una clave y los valores necesarios. El Reducer suma, promedia, busca extremos o entrena el modelo. Las consultas 9 y 10 encadenan dos jobs. Las demás usan un job.

4.2. Diseño de preguntas y consultas

Las preguntas cubren evolución temporal, diferencias territoriales, clases de hospedaje, capacidad, estacionalidad, estadística descriptiva, búsqueda de texto, extremos, cambios entre periodos y participación extranjera. Las cuatro últimas consultas usan la TNOH como etiqueta o variable objetivo. La revisión de antecedentes ayudó a separar las consultas en tres grupos. Primero se midieron patrones descriptivos comparables con los estudios regionales. Luego se usaron jobs encadenados para resumir cambios entre periodos. Finalmente, se compararon modelos sencillos con una división temporal, porque entrenar con años pasados y evaluar 2024 y 2025-I representa mejor el uso real que una división aleatoria.

4.3. Flujo de ejecución

El orquestador ejecuta las consultas en orden y se detiene ante el primer error. Cada salida quedó en su carpeta de resultados, junto al JAR. También se creó un resumen TSV con clase principal, estado, ruta HDFS y artefactos locales.

4.4. Métricas de evaluación

Para clasificación se usaron accuracy, precisión, recall, F1 y log loss. Para regresión se usaron MAE, RMSE y R^2 . El entrenamiento tomó 1 500 filas TT/TT de 2019 a 2023. La prueba usó 300 filas de 2024 y la validación reciente usó 150 filas de 2025-I. La mediana de TNOH del entrenamiento fue 21.71 y definió las clases alta y baja. La estandarización se calculó solo con entrenamiento.

5. Consultas MapReduce

En cada consulta se indican el tipo, la pregunta, los campos, la solución, los archivos de código, la entrada, una salida real, una interpretación y una limitación. La entrada fue el CSV consolidado en todos los casos.

5.1. Resumen de consultas y cantidad de etapas

El Cuadro 2 permite distinguir una consulta de un solo job de una consulta encadenada. “2 jobs” significa que Hadoop ejecuta dos pares Mapper–Reducer consecutivos y que la salida del primero se convierte en la entrada del segundo. En este proyecto, solo las consultas 9 y 10 tienen dos Mappers diferentes.

Cuadro 2: Consultas, tipo de análisis y estructura MapReduce

N.	Consulta	Tipo	Estructura
1	Arribos nacionales y extranjeros	Suma anual por origen	1 Mapper + 1 Reducer
2	TNOH por departamento y año	Promedio agrupado	1 Mapper + 1 Reducer
3	TNOH y TNOG por clase	Comparación de grupos	1 Mapper + 1 Reducer
4	Empleo por cada 100 habitaciones	Razón de sumas	1 Mapper + 1 Reducer
5	Estacionalidad mensual	Suma por mes	1 Mapper + 1 Reducer
6	Estadísticas de TNOH	Estadística descriptiva global	1 Mapper + 1 Reducer
7	Búsqueda de subtexto	Filtro literal	Solo Mapper
8	Mínimo y máximo de TNOH	Extremos de promedios	1 Mapper + 1 Reducer
9	Cambio 2019–2024	Comparación temporal encadenada	2 jobs: 2 Mapper + 2 Reducer
10	Participación extranjera	Agregación mensual y por periodo	2 jobs: 2 Mapper + 2 Reducer
11	Gaussian Naive Bayes	Clasificación supervisada	1 Mapper + 1 Reducer
12	Regresión logística	Clasificación supervisada	1 Mapper + 1 Reducer
13	Regresión lineal	Regresión supervisada	1 Mapper + 1 Reducer
14	Regresión ridge	Regresión regularizada L2	1 Mapper + 1 Reducer

5.2. Consultas que emplean dos o más campos

Este bloque corresponde al ítem 1 de la práctica. Incluye cinco consultas que combinan al menos dos columnas del dataset.

5.2.1. Consulta 1: arribos nacionales y extranjeros

Tipo y pregunta. Agregación por año. ¿Cómo evolucionaron los arribos nacionales y extranjeros? Se usaron ANIO, TOTAL_ARRIBOS_NAC y TOTAL_ARRIBOS_EXT.

Solución y código. ArrivalsMapper omite la cabecera, convierte cada línea en OccupabilityRecord y acepta únicamente registros TT/TT. Su clave es el año y su valor intermedio contiene arribos_nacionales,arribos_extr El shuffle reúne bajo una misma clave todas las filas mensuales y departamentales de un año. ArrivalsReducer recorre esos pares, mantiene dos acumuladores independientes y emite el total anual de cada origen. Es un solo job; no hay un segundo Mapper.

Input y output.

```
2019 national=54897127 foreign=8264767
2020 national=34588588 foreign=1778619
2024 national=50875669 foreign=6690972
2025 national=26355966 foreign=3610473
```

Interpretación y limitación. Se observó una caída fuerte en 2020 y una recuperación posterior. En 2024 los arribos nacionales quedaron cerca de 2019, pero los extranjeros siguieron por debajo. El valor de 2025 solo incluye seis meses y no se puede comparar directamente con un año completo.

5.2.2. Consulta 2: TNOH por departamento y año

Tipo y pregunta. Promedio agrupado. ¿Cuál fue la TNOH media de cada departamento por año? Se usaron año, departamento y TNOH.

Solución y código. `OccupancyMapper` descarta la cabecera y las filas que no son TT/TT. Emite como clave compuesta `ANIO;DEPARTAMENTO` y como valor `TNOH,1`; el uno permite contar cuántos meses entran al promedio. `OccupancyReducer` acumula la TNOH y el contador de cada grupo y emite `suma/cantidad`. El resultado es una media aritmética mensual, no una tasa recalculada ni ponderada por habitaciones.

Input y output.

```
2019;CALLAO mean_tnoh=40.9142
2020;CALLAO mean_tnoh=25.7092
2024;CALLAO mean_tnoh=41.5950
2024;LIMA mean_tnoh=23.5750
2024;LORETO mean_tnoh=12.9917
```

Interpretación y limitación. Callao llegó a 41.5950 en 2024, mientras Loreto obtuvo 12.9917. El promedio resume meses con tamaños de oferta distintos y no equivale a una media ponderada por habitaciones.

5.2.3. Consulta 3: TNOH y TNOC por clase

Tipo y pregunta. Comparación por grupos. ¿Cómo cambian la TNOH y la TNOC según la clase? Se usaron año, clase y ambos porcentajes. Se tomó la categoría total y se excluyó la clase total.

Solución y código. `OccupancyMapper` selecciona filas cuyo total de categoría es TT, pero rechaza el total global TT/TT. Así queda una observación por clase, departamento y mes sin mezclar las categorías de estrellas. Emite `ANIO;CLASE` como clave y `TNOH,TNOC,1` como valor. `OccupancyReducer` mantiene dos sumas y un único contador y produce las medias mensuales de ambas tasas para cada año y clase.

```
2019;HOTEL mean_tnoh=31.2412 mean_tnoc=26.3323
2020;HOTEL mean_tnoh=21.1950 mean_tnoc=16.1440
2024;HOTEL mean_tnoh=32.4184 mean_tnoc=27.6639
2024;RESORT mean_tnoh=46.7742 mean_tnoc=48.2812
```

Interpretación y limitación. Resort mantuvo medias altas, mientras Hotel cayó en 2020 y luego se recuperó. Las clases no tienen la misma cantidad de establecimientos, así que la salida no compara tamaños totales.

5.2.4. Consulta 4: empleo por capacidad

Tipo y pregunta. Razón entre campos. ¿Cuántos empleos existen por cada cien habitaciones en cada departamento y año? Se usaron año, departamento, empleo y habitaciones.

Solución y código. `CapacityMapper` filtra TT/TT, forma la clave `ANIO;DEPARTAMENTO` y envía `TOTAL_EMPLEO` junto con `NUMERO_HABITACIONES`. `CapacityReducer` suma por separado el empleo y las habitaciones de todos los meses y recién después calcula $100 \times \text{empleo} / \text{habitaciones}$. Esta razón de sumas evita promediar razones mensuales con igual peso; si el denominador fuera cero, devuelve 0.

```
2019;LIMA employment_per_100_rooms=74.3232
2024;LIMA employment_per_100_rooms=54.3562
2024;CALLAO employment_per_100_rooms=28.4118
2024;TACNA employment_per_100_rooms=15.0163
```

Interpretación y limitación. Lima presentó más empleos por cada cien habitaciones que Callao y Tacna. La razón no mide calidad del empleo ni horas trabajadas; solo relaciona los totales informados.

5.2.5. Consulta 5: estacionalidad mensual

Tipo y pregunta. Suma por mes. ¿Qué meses concentran más arribos y pernoctaciones? Se usaron mes, arribos y pernoctaciones TT/TT.

Solución y código. `SeasonalityMapper` conserva TT/TT y usa el mes con dos dígitos como clave, sin incluir el año. Su valor contiene `TOTAL_ARRIBOS,TOTAL_PERNOCT`. El shuffle junta, por ejemplo, todos los eneros de todos los años y departamentos. `SeasonalityReducer` suma por separado personas llegadas y noches ocupadas. Por eso la salida mide volumen acumulado por mes calendario, no el promedio de un enero típico.

```
01 arrivals=33585654 overnight_stays=43787718
02 arrivals=32795352 overnight_stays=42942772
07 arrivals=26760124 overnight_stays=35417748
09 arrivals=26098980 overnight_stays=33472076
```

Interpretación y limitación. Enero y febrero tuvieron los mayores acumulados, mientras septiembre tuvo el menor valor. Enero a junio incluyen siete años, pero julio a diciembre solo seis. La suma no está balanceada.

5.3. Medidas estadísticas

Esta consulta corresponde al ítem 2 de la práctica.

5.3.1. Consulta 6: media, mediana y desviación estándar de TNOH

Tipo y pregunta. Estadística descriptiva. ¿Cuál es la cantidad, media, mediana y desviación estándar poblacional de TNOH?

Solución y código. `StatisticsMapper` filtra TT/TT y envía cada TNOH con la clave constante TNOH; esta decisión obliga a que todos los valores lleguen al mismo grupo. `StatisticsReducer` guarda las 1 950 observaciones, calcula la media y la varianza poblacional en una pasada con el algoritmo incremental de Welford, y después ordena la lista para hallar la mediana. La desviación usa divisor N , no $N - 1$.

```
TNOH count=1950 mean=21.7433 median=21.3400 stddev=7.0678612717
```

Interpretación y limitación. La media y mediana quedaron cerca, y la desviación fue 7.0679 puntos. Un reducer pudo reunir los valores porque el dataset es pequeño. Para más datos convendría un método distribuido de cuantiles.

5.4. Búsqueda de subtexto

Esta consulta corresponde al ítem 3 de la práctica.

5.4.1. Consulta 7: búsqueda sobre campos de texto

Tipo y pregunta. Filtro por subtexto. ¿Qué registros contienen la palabra hotel en clase, categoría o departamento? La búsqueda no distingue mayúsculas, pero conserva los acentos.

Solución y código. En `configure`, `SearchMapper` recibe el término pasado al Driver y lo transforma a minúsculas. Para cada fila omite la cabecera, parsea clase, categoría y departamento, y busca el término con `contains` después de convertir cada campo a minúsculas. Si cualquiera coincide, emite como clave el desplazamiento en bytes de la línea y como valor la fila canónica completa. El Driver fija cero reducers: es una consulta *map-only*, porque cada coincidencia puede decidirse de forma independiente.

```
2424268 ;2022;06;01;HOTEL;01;1 ESTRELLA;AMAZONAS;010000;1;20;42;...
2424367 ;2022;06;01;HOTEL;01;1 ESTRELLA;PIURA;200000;2;40;77;...
```

Interpretación y limitación. Se obtuvieron 11 525 coincidencias porque HOTEL aparece como clase en muchos registros. La búsqueda es literal; no aplica sinónimos, ranking ni normalización de acentos.

5.5. Valores máximos y mínimos

Esta consulta corresponde al ítem 4 de la práctica.

5.5.1. Consulta 8: extremos de TNOH por año

Tipo y pregunta. ¿Qué departamentos tuvieron la menor y mayor TNOH media de cada año? Se usaron año, departamento y TNOH.

Solución y código. `ExtremesMapper` conserva TT/TT, emite el año como clave y DEPARTAMENTO, TNOH como valor. De este modo, un reducer recibe todos los departamentos y meses del mismo año. Allí, `ExtremesReducer` usa un acumulador de suma y cantidad por departamento, calcula cada media y recorre las medias para elegir mínimo y máximo. Aunque hay dos niveles lógicos —promediar y elegir extremos—, ambos ocurren dentro del mismo reducer; NO es una consulta de dos jobs.

```
2019 min=TACNA:20.7342 max=CALLAO:40.9142
2020 min=TACNA:9.6425 max=CALLAO:25.7092
2024 min=LORETO:12.9917 max=CALLAO:41.5950
2025 min=AYACUCHO:14.1217 max=CALLAO:43.9917
```

Interpretación y limitación. Callao fue el máximo en seis años; en 2021 Piura quedó arriba. Los extremos usan promedios simples y 2025 solo cubre el primer semestre.

5.6. Consultas con MapReduce encadenado

Este bloque corresponde al ítem 5 de la práctica. Cada consulta usa dos jobs MapReduce enlazados.

5.6.1. Consulta 9: cambio de ocupación entre 2019 y 2024

Tipo y pregunta. Dos jobs. ¿Cuántos puntos porcentuales cambió la TNOH media de cada departamento entre 2019 y 2024?

Solución y código. Es la primera consulta con dos Mappers. En el job 1, `AnnualMapper` conserva únicamente TT/TT de 2019 y 2024, y emite `ANIO;DEPARTAMENTO ->TNOH,1`; `AnnualReducer` produce la media de cada año y departamento. En el job 2, `ChangeMapper` lee esa salida intermedia, cambia la clave a departamento y conserva `ANIO,media` como valor. Así, `ChangeReducer` recibe juntos los dos años, comprueba que ambos existan y emite $media_{2024} - media_{2019}$ en puntos porcentuales. El Driver crea la ruta intermedia, ejecuta los jobs en orden y la elimina al final.

```
LA LIBERTAD 2019=30.7350 2024=14.7108 change_pp=-16.0242
CALLAO 2019=40.9142 2024=41.5950 change_pp=0.6808
CUSCO 2019=26.9108 2024=28.2725 change_pp=1.3617
UCAYALI 2019=25.1383 2024=30.1825 change_pp=5.0442
```

Interpretación y limitación. La mayor caída fue Apurímac con $-18,7325$ puntos y la mayor subida fue Ucayali con 5,0442. La diferencia describe una asociación temporal y no prueba la causa.

5.6.2. Consulta 10: participación de arribos extranjeros

Tipo y pregunta. Dos jobs. ¿Qué porcentaje de los arribos fue extranjero en cada departamento y periodo?

Solución y código. También usa dos Mappers. En el job 1, `MonthlyMapper` filtra TT/TT con arribos mayores que cero, asigna el año a uno de cuatro periodos y calcula $100 \times arribos_extranjeros / arribos_totales$. Emite una clave que identifica departamento, periodo, año y mes. `MonthlyReducer` consolida posibles duplicados mensuales y deja una participación por mes. En el job 2, `PeriodMapper` elimina año y mes de la clave y reagrupa por departamento-periodo; `PeriodReducer` promedia las participaciones mensuales. Es una media de porcentajes mensuales, no una participación calculada sobre los totales acumulados del periodo.

```
CUSCO;PRE_2019 mean_foreign_share=63.3747
CUSCO;IMPACT_2020_2021 mean_foreign_share=20.5234
CUSCO;RECOVERY_2022_2024 mean_foreign_share=58.3507
CUSCO;CURRENT_2025_H1 mean_foreign_share=67.4504
```

Interpretación y limitación. En Cusco se observó una caída fuerte en 2020–2021 y recuperación posterior. Se promediaron porcentajes mensuales sin ponderar por arribos; no equivale a dividir los acumulados del periodo.

6. Consultas mediante Modelos de Clasificación

Este bloque corresponde al ítem 6 de la práctica y compara dos modelos de clasificación con la misma partición temporal.

6.1. Consulta 11: Gaussian Naive Bayes

Pregunta, campos y código. Se clasificó la TNOH como alta o baja. Se usaron mes, establecimientos, habitaciones, camas, arribos por habitación, pernoctaciones por cama y empleo por establecimiento. El Mapper preparó filas TT/TT y el Reducer entrenó con 2019–2023. El Driver es `NaiveBayesClassificationDriver`. En detalle, `ModelMapper` filtra TT/TT y codifica cada fila como año, TNOH y siete características numéricas bajo la clave constante `MODEL`. `ModelReducer` reúne el conjunto, calcula la mediana de TNOH solo con 2019–2023, crea la etiqueta alta/baja, entrena Gaussian Naive Bayes con esos años y evalúa por separado 2024 y 2025-I. El Mapper no entrena: prepara ejemplos; el entrenamiento está centralizado en un reducer.

```
TEST_2024 samples=300 accuracy=0.703333 precision=0.945946 recall=0.286885 f1=0.440252 log_loss=0.947039
VALIDATION_2025_H1 samples=150 accuracy=0.726667 precision=1.000000 recall=0.305085 f1=0.467532 log_loss=0.854541
```

Interpretación y limitación. Se obtuvo precisión alta, pero recall bajo. El modelo dejó sin detectar muchas observaciones altas. La independencia condicional es una limitación para indicadores relacionados.

6.2. Consulta 12: regresión logística

Pregunta, campos y código. Se respondió la misma pregunta con las mismas siete variables, estandarizadas solo con entrenamiento. El Reducer entrenó 200 iteraciones con tasa 0.1. El Driver es `LogisticClassificationDriver`. `ModelMapper` realiza la misma selección TT/TT y la misma codificación que la consulta 11, para que la comparación sea justa. `ModelReducer` calcula la etiqueta con la mediana del entrenamiento, estandariza las siete características usando únicamente 2019–2023, ajusta la regresión logística durante 200 iteraciones y emite la matriz de confusión y las métricas de 2024 y 2025-I. Es un solo job con un reducer de entrenamiento, no dos etapas MapReduce.

```
TEST_2024 samples=300 accuracy=0.840000 precision=0.836364 recall=0.754098 f1=0.793103 log_loss=0.498765
VALIDATION_2025_H1 samples=150 accuracy=0.800000 precision=0.871795 recall=0.576271 f1=0.693878 log_loss=0.474774
```

Interpretación y limitación. Se obtuvo un equilibrio más claro entre precisión y recall. El entrenamiento se centralizó en un reducer porque solo hubo 1 500 filas; esto no representa entrenamiento distribuido a gran escala.

6.3. Comparación de modelos de clasificación

Cuadro 3: Resultados de clasificación con salidas reales

Periodo	Modelo	Accuracy	Precisión	Recall	F1	Log loss
2024	Naive Bayes	0.7033	0.9459	0.2869	0.4403	0.9470
2024	Logística	0.8400	0.8364	0.7541	0.7931	0.4988
2025-I	Naive Bayes	0.7267	1.0000	0.3051	0.4675	0.8545
2025-I	Logística	0.8000	0.8718	0.5763	0.6939	0.4748

La regresión logística tuvo mayor accuracy, recall y F1, y menor log loss. Naive Bayes tuvo mayor precisión, pero perdió muchos positivos. Con estos datos, la regresión logística dejó el mejor resultado general.

7. Consultas mediante Modelos de Regresión

Este bloque corresponde al ítem 7 de la práctica y compara dos modelos de regresión con las mismas variables y periodos de evaluación.

7.1. Consulta 13: regresión lineal

Pregunta, campos y código. Se estimó la TNOH con las mismas siete características y la misma separación temporal. El Reducer ajustó una regresión sin regularización. El Driver es `LinearRegressionDriver`. `ModelMapper` convierte cada fila TT/TT en año, TNOH objetivo y las siete características, siempre bajo la clave `MODEL`. `ModelReducer` separa temporalmente los ejemplos, ajusta el escalado solo con entrenamiento, resuelve la regresión con $\lambda = 0$ y aplica el modelo a 2024 y 2025-I para acumular MAE, RMSE y R^2 . El reducer también emite intercepto y coeficientes.

```
TRAINING samples=1500
TEST_2024 samples=300 mae=3.192221 rmse=4.092002 r_squared=0.643537
VALIDATION_2025_H1 samples=150 mae=3.020832 rmse=3.927314 r_squared=0.662158
```

Interpretación y limitación. Se obtuvo un MAE cercano a tres puntos. El R^2 fue 0.6435 en 2024 y 0.6622 en 2025-I. El modelo solo representa relaciones lineales.

7.2. Consulta 14: regresión ridge

Pregunta, campos y código. Se estimó la misma variable con regularización L2, $\lambda = 1$, sin regularizar el intercepto. Se usaron las clases internas de `RidgeRegressionDriver`. El flujo de `ModelMapper` y `ModelReducer` es deliberadamente igual al de la consulta 13; la única diferencia del ajuste es $\lambda = 1$. El reducer estandariza con entrenamiento, aplica penalización L2 a los coeficientes sin penalizar el intercepto y calcula las mismas métricas en los mismos periodos. Mantener idénticos datos y flujo permite atribuir la diferencia observada a la regularización y no a otro filtro.

TRAINING samples=1500
 TEST_2024 samples=300 mae=3.194316 rmse=4.072566 r_squared=0.646915
 VALIDATION_2025_H1 samples=150 mae=3.013257 rmse=3.879584 r_squared=0.670320

Interpretación y limitación. Ridge redujo un poco el RMSE y mejoró el R^2 . Solo se evaluó un valor de λ , así que no se puede afirmar que sea la mejor regularización posible.

7.3. Comparación de modelos de regresión

Cuadro 4: Resultados de regresión con salidas reales

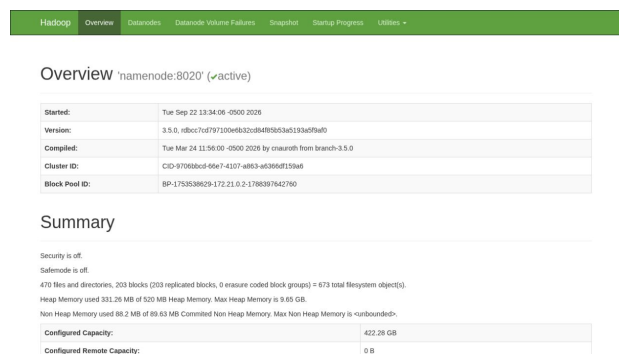
Periodo	Modelo	MAE	RMSE	R^2
2024	Lineal	3.1922	4.0920	0.6435
2024	Ridge	3.1943	4.0726	0.6469
2025-I	Lineal	3.0208	3.9273	0.6622
2025-I	Ridge	3.0133	3.8796	0.6703

La diferencia fue pequeña. Ridge quedó mejor en RMSE y R^2 . En MAE, el lineal fue apenas mejor en 2024 y ridge fue mejor en 2025-I.

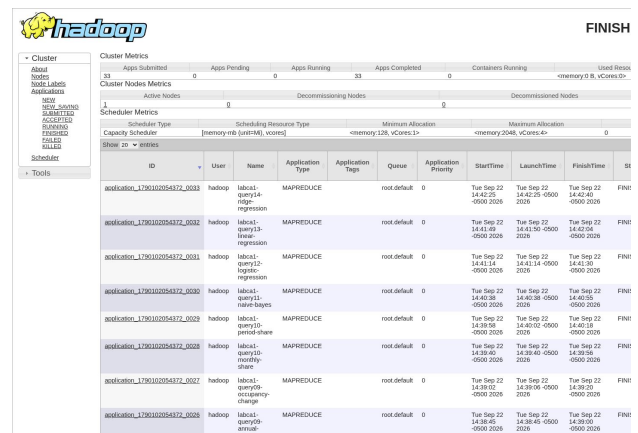
8. Monitoreo de Recursos

8.1. Recursos observados durante la ejecución

Las interfaces mostraron un NameNode activo, un NodeManager saludable y 33 aplicaciones completadas. La Figura 1 reúne HDFS y YARN. La Figura 2 muestra un job real de la consulta 14.

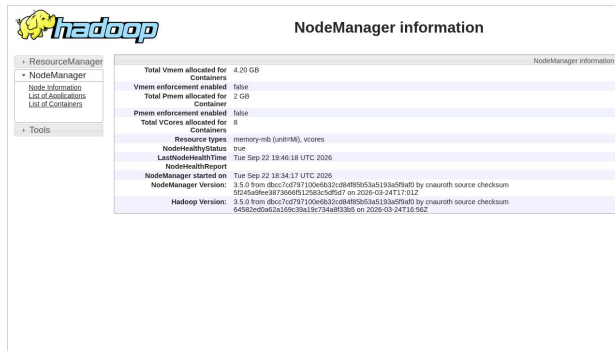


(a) NameNode activo y resumen de HDFS.

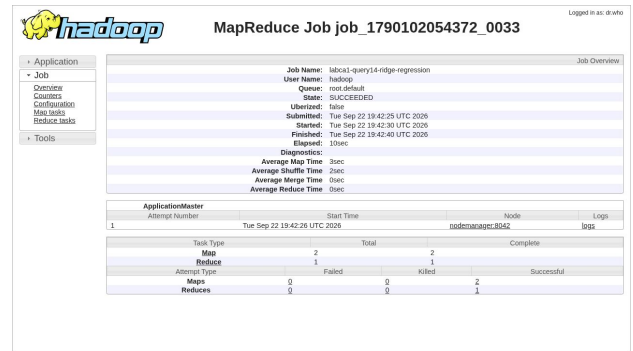


(b) Aplicaciones finalizadas en YARN.

Figura 1: Estado de los servicios principales de Hadoop.



(a) Recursos y estado del NodeManager.



(b) Consulta 14 con estado SUCCEEDED.

Figura 2: Recursos del nodo y detalle de una ejecución.

8.2. Consumo de CPU

La medición se tomó el 22 de septiembre de 2026 a las 14:41:56, mientras se ejecutaba la consulta 13. El NodeManager registró 1103.66 % de CPU. Docker mide 100 % por núcleo, así que el valor indica uso de varios núcleos.

8.3. Consumo de memoria

El NodeManager usó 1.193 GiB. El NameNode usó 992.1 MiB, el ResourceManager 475.5 MiB, el History-Server 346.9 MiB y el DataNode 335.5 MiB. Es una medición puntual, no un promedio.

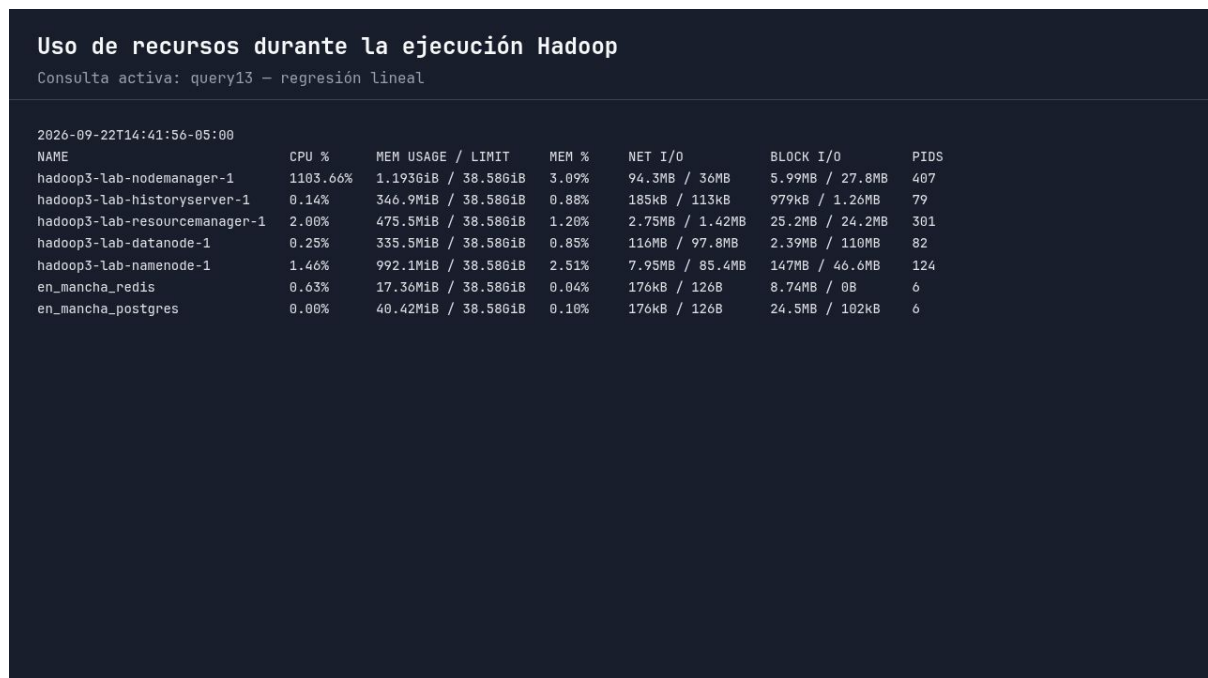


Figura 3: Uso de recursos durante la consulta 13.

8.4. Tiempo de ejecución

JobHistory registró 10 segundos para el job ridge de la consulta 14: empezó a las 19:42:30 UTC y terminó a las 19:42:40. No incluye copia del código, compilación ni exportación.

8.5. Análisis del comportamiento de Hadoop

Los trabajos pequeños tienen un costo fijo al iniciar mapas, shuffle y reducir. El clúster procesó el CSV sin fallos y dejó trazas. El dataset y el único NodeManager prueban el flujo completo, pero no demuestran escalabilidad horizontal.

9. Resultados y Discusión

9.1. Resultados de las consultas MapReduce

Las salidas mostraron la caída de 2020, diferencias entre departamentos y una recuperación desigual. Callao presentó TNOH alta en varios años. En el cambio 2019–2024, Apurímac tuvo la mayor caída y Ucayali la mayor subida. Cusco mostró una caída de participación extranjera en 2020–2021 y recuperación posterior.

Las cifras se contrastaron con Python en modo de solo lectura. Los totales de las consultas 1 y 5 y las estadísticas de la consulta 6 coincidieron. También se confirmaron 1 500 muestras de entrenamiento, 300 de prueba y 150 de validación. Sumar todos los niveles cuadruplicaba aproximadamente algunos totales; el filtro TT/TT evitó ese doble conteo.

9.2. Resultados de clasificación

La regresión logística fue más equilibrada. En 2024 obtuvo F1 de 0.7931 frente a 0.4403 de Naive Bayes. En 2025-I obtuvo 0.6939 frente a 0.4675. Naive Bayes tuvo precisión alta, pero recall bajo.

9.3. Resultados de regresión

Ridge mejoró el RMSE de 4.0920 a 4.0726 en 2024 y de 3.9273 a 3.8796 en 2025-I. También elevó un poco el R^2 . La mejora fue pequeña.

9.4. Discusión general

Se logró responder las catorce consultas con una entrada y una ejecución repetible. Las consultas encadenadas mostraron cómo una salida intermedia alimenta otro job. En los modelos se usó una división temporal para entrenar con el pasado y evaluar periodos posteriores.

Las limitaciones principales son el dataset pequeño, un solo nodo, algunos promedios no ponderados y el semestre incompleto de 2025. Además, los modelos se entrenaron en un reducer. Fue suficiente para 1 950 filas TT/TT, pero no sería adecuado para un modelo muy grande.

10. Conclusiones

Se obtuvo un CSV canónico de 38 730 registros sin modificar los siete archivos originales. El filtro TT/TT evitó doble conteo. Las catorce consultas terminaron correctamente y conservaron resultado y JAR.

Se observó una caída de varios indicadores en 2020 y una recuperación diferente entre departamentos. Las consultas encadenadas compararon 2019 con 2024 y agruparon la participación extranjera por periodos.

La regresión logística obtuvo el mejor equilibrio en clasificación. Ridge mejoró ligeramente RMSE y R^2 . Estos resultados describen este dataset y no prueban causas ni garantizan el mismo desempeño en otros periodos.

Finalmente, se logró comprobar HDFS, YARN y JobHistory en un nodo. Las capturas muestran trabajos y consumo reales. El entorno sirve para el laboratorio, pero no demuestra escalabilidad horizontal.

A. Anexo: Código fuente Java y archivos JAR

El código está en `queries/common/src` y `queries/query01` a `queries/query14`. Cada consulta conserva su JAR en `resultados/queryXX/queryXX.jar`.

B. Anexo: Inputs y outputs de las consultas

La entrada es `data/Indicadores_ocupabilidad_2019_2025.csv`. Cada salida completa está en `queries/queryXX/resultados/queryXX/resultado.txt`. `queries/execution-summary.tsv` registra las catorce ejecuciones.

C. Anexo: Docker Compose y configuración de Hadoop

La definición está en `hadoop3/compose.yaml`. Los scripts están en `hadoop3/scripts/`.

D. Anexo: Documentación adicional

Cada consulta incluye un README. Las evidencias se documentan en `labs/labca1/images/README.md`.

Referencias

- [1] Ministerio de Comercio Exterior y Turismo, “Indicadores de ocupabilidad [conjunto de datos].” Plataforma Nacional de Datos Abiertos (PNDA), Presidencia del Consejo de Ministros, Perú, 2021. Publicado el 31 de diciembre de 2021; última actualización del metadato: 4 de agosto de 2026. 11 recursos CSV (2015–2025 S1), diccionario de datos, metadatos y ficha técnica. Licencia Open Data Commons Attribution (ODC-BY). Identificador CKAN: dbf6bc9c-cabe-47f7-b052-d11cfdd4d9b0. Fuente primaria: Encuesta Mensual de Turismo para Establecimientos de Hospedaje (DGIETA-MINCETUR).
- [2] Ministerio de Economía y Finanzas, “Formulación de proyectos de inversión para la creación, mejoramiento, ampliación y recuperación de servicios turísticos públicos: Ficha técnica simplificada,” tech. rep., Ministerio de Economía y Finanzas del Perú, Dirección General de Inversión Pública, Lima, Perú, Nov. 2021. VENTANA EXTENDIDA 2021. Capacitación oficial del MEF que instruye el uso de los Indicadores mensuales de ocupabilidad de establecimientos de hospedaje del MINCETUR como población de referencia para la estimación de demanda en proyectos de inversión pública turística.
- [3] Instituto Nacional de Estadística e Informática, “Resolución jefatural n.º 037-2024-inei: Autorizan ejecución de la encuesta “estadística mensual de turismo 2024 para establecimientos de hospedaje”.” Diario Oficial El Peruano, Feb. 2024.
- [4] H. Taipei Pari and J. G. Laura Huamán, “Desestacionalización de la serie de tiempo llegada mensual de turistas y la oferta de hospedaje en la región huancavelica, periodo 2003–2019,” tesis para optar el título profesional de economista, Universidad Nacional de Huancavelica, Huancavelica, Perú, 2021. VENTANA EXTENDIDA 2021. Construye su base de datos a partir de los indicadores mensuales de ocupabilidad de establecimientos de hospedaje colectivo de la región Huancavelica (2003–2019), procesada en EViews 10. Regresión lineal multivariada con variables dicotómicas estacionales y MCO.
- [5] M. Ñahuincopa Huarancca and W. M. Rojas Romero, “La demanda turística y la capacidad hotelera en la región huancavelica, período 2010–2019,” tesis de grado, Universidad Nacional de Huancavelica, Huancavelica, Perú, 2022. Modelo lineal multivariado; la capacidad hotelera muestra un efecto limitado sobre la demanda turística y la elasticidad ingreso resulta inelástica.
- [6] J. Bravo, R. Alarcón, C. Valdivia, and O. Serquén, “Application of machine learning techniques to predict visitors to the tourist attractions of the moche route in peru,” *Sustainability*, vol. 15, no. 11, p. 8967, 2023. Consume CSV de la PNDA del Perú más TripAdvisor y Google Trends; 1 534 registros (2011–2022); cuatro algoritmos de regresión.
- [7] A. P. Viteri Gonzales and J. A. Beltrán García, “Solución de big data para el análisis de los datos abiertos de minsa y cenares para el monitoreo y control de la emergencia sanitaria covid-19 bajo el ecosistema de apache hadoop y microsoft azure,” tesis de grado, Universidad Privada Antenor Orrego, Trujillo, Perú, 2023. Arquitectura Apache Hadoop + Azure sobre datos abiertos de la PNDA del Perú; dominio salud.

- [8] M. Mariani and R. Baggio, “Big data and analytics in hospitality and tourism: a systematic literature review,” *International Journal of Contemporary Hospitality Management*, vol. 34, no. 1, pp. 231–278, 2022. Taxonomía de fuentes de big data en turismo: datos administrados vs. generados por usuarios.